WEB DEVELOPMENT-I

Made By : **KASHIF SAYYAD**

UNIT 1 — HTML5 FUNDAMENTALS

■ HTML Document Structure and Common Elements

HTML (HyperText Markup Language) is the language in which every web page is written: the text of the page marked with **tags** that tell the browser how to show it. An HTML5 document has a fixed skeleton: the **<!DOCTYPE html>** declaration, the **<html>** root, the **<head>** (title, meta data, links to CSS and scripts — not shown on the page) and the **<body>** (everything visible). Tags are mostly written in pairs: an opening tag, the content, a closing tag; some tags are self-closing, e.g. **
, <hr>, <img>**, and tags may carry attributes like *src*, *href* and *class*.

Main heading

This is a paragraph written normally.

bold text and *italic text*

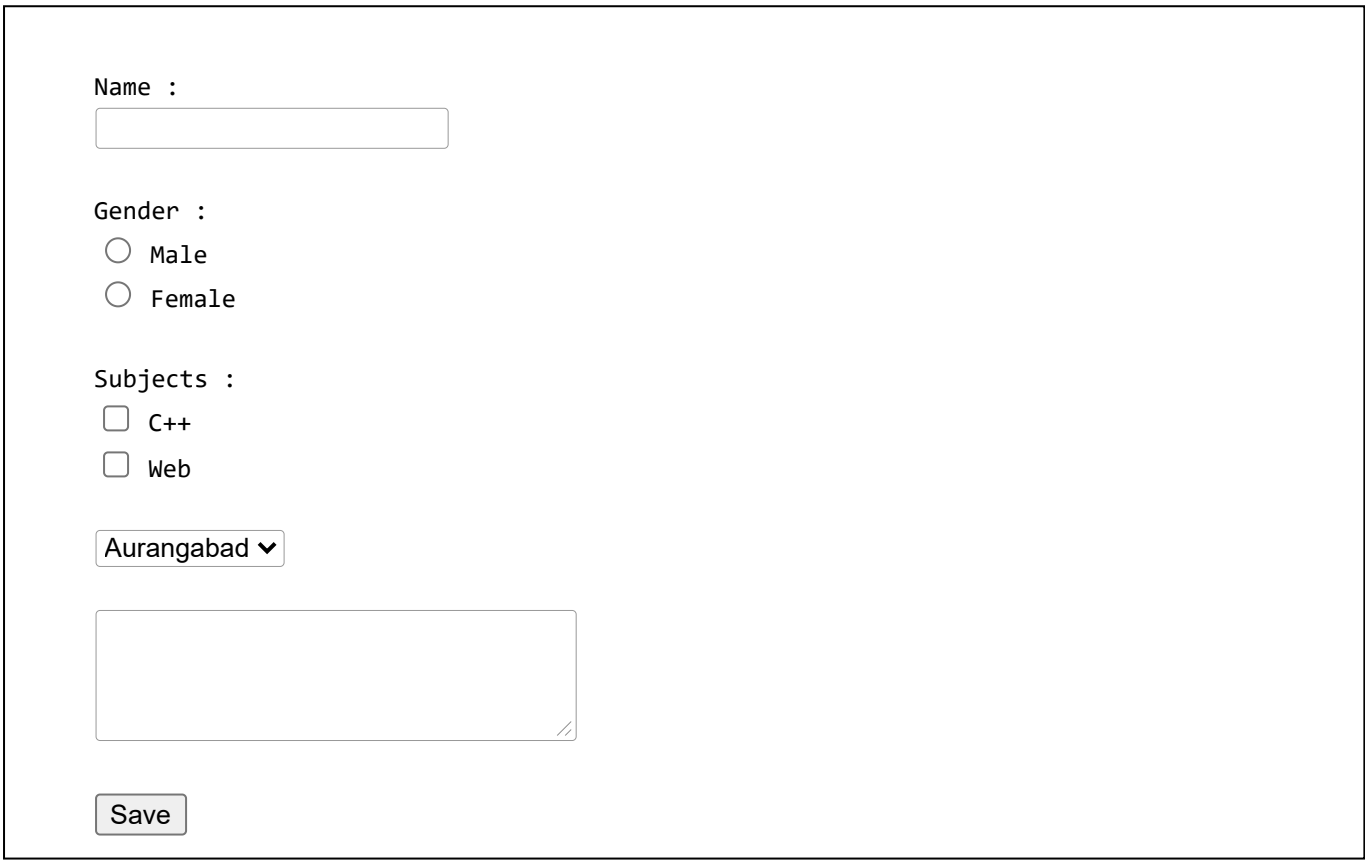
a link

my photo

The **commonly used elements** are: headings **<h1> ... <h6>** (six levels of importance), the paragraphs **<p>**, line break **
, horizontal rule **<hr>, emphasis **<b> <i> <u>**, links **<a>**, images **<img>**, lists **<ul>**/**<ol>** with **<li>**, and the table tags **<table> <tr> <th> <td>**.

■ HTML Forms and Form Elements

A form is the HTML part which collects data from the user and either sends it to the server or handles it with JavaScript; everything goes inside the **<form>** tag with the attributes **action** (where the data goes) and **method** (get or post). The workhorse is the **<input>** tag whose **type** attribute makes it behave differently: text, password, email, number, date, radio, checkbox, file, submit, reset and button. The other form elements are **<label>**, **<textarea>** for long text, **<select>** with **<option>** for dropdowns, **<fieldset>** and **<legend>** for grouping, and **<button>**.



■ Horizontal Rules and Semantic Text Elements

The tag **<hr>** draws a **horizontal rule** — a line across the page used to separate sections or change of a topic; it is a void tag and can be styled with CSS instead of old attributes like *size* and *width*. The HTML **text-formatting elements** are: **<b>** draws notice to the text with bold, **** states importance (also bold, but with meaning for the screen-reader and the search engines), **<i>** and **** for italic and emphasis, and **<mark>** highlights the text in yellow by default.

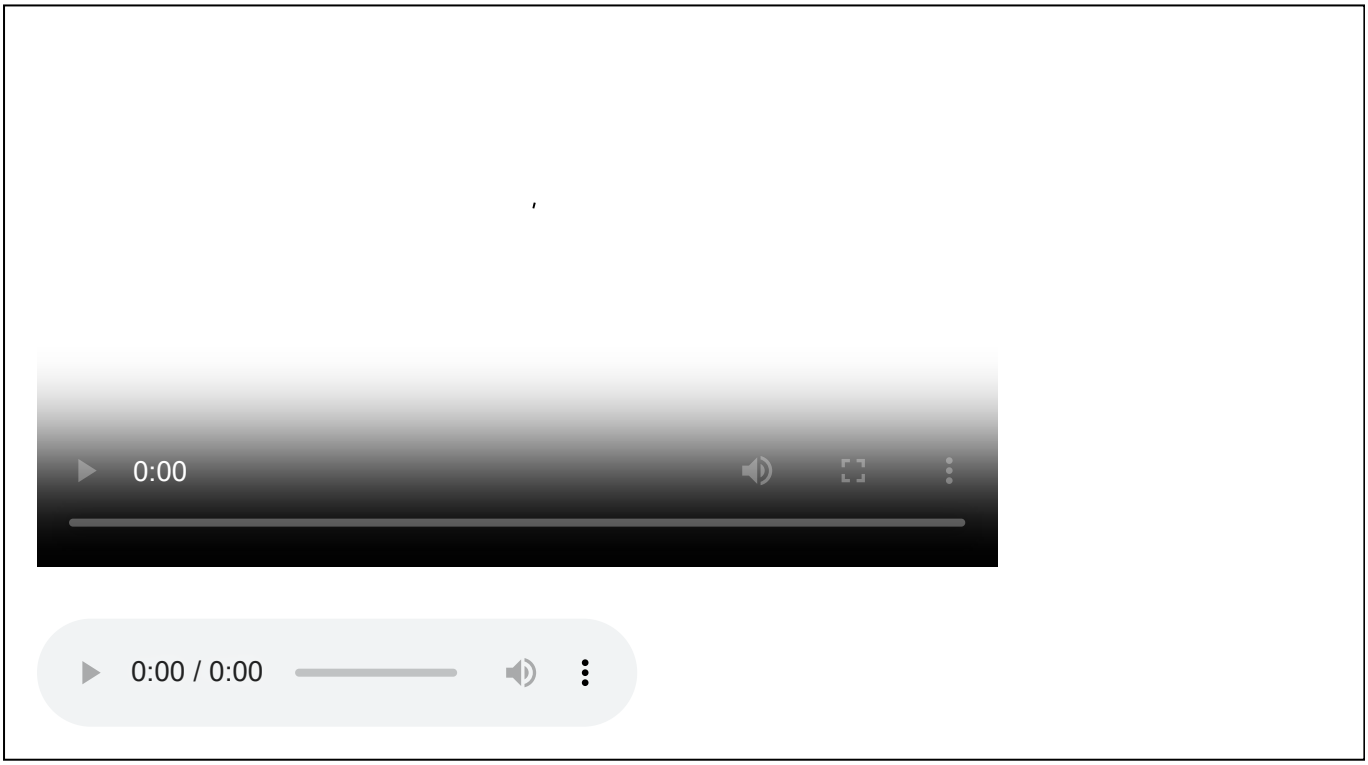
This is the **semantic** idea of HTML5: markup should describe the **meaning of the content**, not the appearance. **<b>** and **** look the same in the browser, but **** says "this text is important" while **<b>** says only "these letters in bold"; the same pair **<i>** / **** distinguish appearance from stress. HTML5 also gives the page-structure tags **<header> <nav> <section> <article> <aside> <footer>** that replace many anonymous ** **s and make the document readable to the browser, the screen-reader and the crawler.

Tag	Visual effect	Semantic meaning
<b>	bold	stir attention, no special importance
	bold	strong importance
<i>	italic	technical term, foreign phrase, thought
	italic	stress emphasis
<mark>	yellow background	highlighted / relevant text
<hr>	horizontal line	thematic break between topics

UNIT 2 — HTML5 MULTIMEDIA AND GRAPHICS

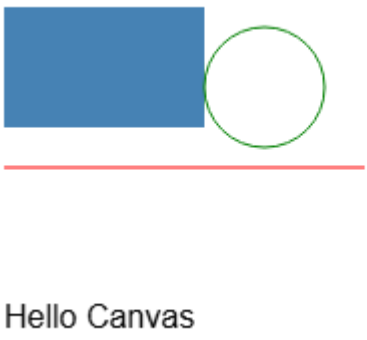
■ Displaying Video with HTML5 Media Elements

Before HTML5, video and audio needed plug-ins like Flash; HTML5 introduced the native **<video>** and **<audio>** tags which the browser itself understands. The **<video>** element places a video with the tag **<video src="movie.mp4" controls>fallback text</video>**; its attributes are: **controls** (show the play, pause, volume bar), **autoplay**, **loop**, **muted**, **poster** (the image shown before the video plays), **preload** and **width/height**. Several **<source>** children may be given — the browser uses the first format it can play (commonly mp4 with H.264, webm, ogg); the text between the tags is shown by browsers which do not support the video at all.



■ The Canvas Element

The **<canvas>** element creates a fixed-size, blank **rectangular drawing area** on which graphics are drawn with **JavaScript** only — the element itself has no drawing power. Its only attributes are **width** and **height** (default 300 by 150); its **2D context**, got by *getContext('2d')*, gives the drawing methods. Everything is drawn in a co-ordinate system whose origin **(0,0)** is the top-left corner with x increasing to the right and y downwards. Canvas is used for charts, games, image editing, and animation: the drawn shapes are not objects — pixels are painted on the canvas, so to change a picture the whole area is cleared and redrawn.



Remember : Difference in one line: **<img>** shows a ready-made picture, **<svg>** keeps shapes as scalable vector objects, while **<canvas>** is a blank pixel board for drawing and erasing with JavaScript — the natural choice for animation and games.

UNIT 3 — INTRODUCTION TO CSS

■ Purpose and Syntax of CSS; Selectors and Declarations

Cascading Style Sheets separate the **presentation** (colours, fonts, spacing, layout) from the HTML content: one stylesheet can style a thousand pages, and the markup remains clean. CSS syntax is a set of **rules**; each rule has a **selector** (what to style) and a **declaration block** inside curly braces; a declaration is a **property : value;** pair, e.g. *color : red;*. The cascade means that, when rules fight, the winner is decided by importance, specificity and the order in the sheet: inline style beats internal, internal beats external (when specificity of the selectors is the same).

- **Element selector** — p { } : styles every <p>.
- **Class selector** — .note { } : styles every element with class="note" (the most used).
- **ID selector** — #top { } : styles the one element with that id.
- **Universal selector** — * { } : selects everything.
- **Grouping selector** — h1, p { } : one rule for several selectors.
- **Descendant selector** — div p { } : every <p> inside a .

```
/* element */
p      { color: #222; line-height: 1.6; }

/* class */
.note  { background: #ffffcc; padding: 6px; }

/* id */
#top   { border-bottom: 2px solid #000; }

/* grouping */
h1, h2 { font-family: Arial, sans-serif; }
```

■ Inline, Internal and External CSS

Way	Where it sits	Syntax sketch	Best use
Inline	in the style attribute of the very element	<p style="color:red;">	one small, one-off change; has the highest priority
Internal	a <style> block in the <head> of the same page	<style> p { } </style>	styling one single page
External	a separate .css file connected with <link>	<link rel="stylesheet" href="style.css">	styling a whole site — the professional way

---

```
inline styling of one paragraph
```

■ Text Alignment, Text Styling, Colours and background-color

The helpful **text properties**: **text-align** (left, right, center, justify), **text-decoration** (underline, line-through, none — removes the underline of links), **text-transform** (uppercase, lowercase, capitalize), **letter-spacing** and **line-height**. The **font properties**: **font-family** (always give a fallback list: Arial, sans-serif), **font-size** (px, em, %), **font-weight** (bold, 300/600/700), **font-style** (italic). **Colours** may be named (red), hexadecimal (#ff0000 /

#f00), rgb(255,0,0), rgba(255,0,0,0.5) or hsl(0, 100%, 50%); the **color** property paints the text and the **background-color** property paints the box behind it.

```
h1 {
    text-align: center;
    text-transform: uppercase;
    font-family: Arial, sans-serif;
}

p {
    text-align: justify;
    line-height: 1.6;
    color: rgb(40, 40, 40);
}

.box {
    background-color: #eef;
    color: navy;
    padding: 8px;
}

a  { text-decoration: none; }
mark { background-color: yellow; }
```

UNIT 4 — CSS BOX MODEL AND POSITIONING

■ The CSS Box Model

Every HTML element is treated by CSS as a rectangular **box**, built from the inside out: **content** (the text or the image), wrapped by **padding** (transparent space inside, between content and border), then the **border** (the drawn line), then the **margin** (transparent space outside, which keeps boxes away from each other). By default width/height refer to the content only; **box-sizing : border-box;** makes width include padding and border, which makes layout arithmetic simple. study the total width of a box = margin-left + border-left + padding-left + width + padding-right + border-right + margin-right.

```
.card {
    width: 300px;
    padding: 16px;          /* space inside  */
    border: 2px solid #333; /* the line      */
    margin: 24px auto;      /* space outside, auto = centred */
    background: #fff;
    box-sizing: border-box; /* 300px now includes padding + border */
}
```

■ Margin and Padding and their Differences

	Padding	Margin
Where	inside the box, between the content and the border	outside the box, between this box and its neighbours
Inherits the background of the element?	yes — the background colour covers the padding	no — margin is always transparent
Negative values	not allowed	allowed (pulls boxes closer)
Auto value	no meaning	auto centres a block horizontally (margin : 0 auto;)
Collapsing	padding never collapses	vertical margins of two touching blocks collapse into one

---

The shorthand runs clockwise from the top: **padding : 10px 20px 30px 40px;** means top 10, right 20, bottom 30, left 40; two values mean vertical then horizontal (10px 20px = 10 top and bottom, 20 left and right); one value is used on all the four sides. The same scales for margin. The specific long names like **padding-left** and **margin-top** are used for one side only.

■ Relative and Absolute Positioning

By default every element is **static**: it falls where the normal flow puts it and the properties top, right, bottom and left do not affect it. **position : relative;** keeps the element in the normal flow (its place is still reserved) but lets it be moved from that spot by top/left/right/bottom; other elements act as if it never moved. **position : absolute;** removes the element from the normal flow and places it at the given co-ordinates **relative to its nearest positioned ancestor** (an ancestor with position other than static — usually the parent which is relative); if no positioned ancestor exists, it is positioned relative to the page itself. The pair *relative parent + absolute child* is the usual way of making overlays, badges and dropdown menus. Other values are **fixed** (pinned to the browser window; does not move when the page is scrolled) and **sticky** (normal, until the scroll pushes it to the given offset, then it sticks).

```
.parent {
    position: relative;          /* anchor for the child */
    width: 300px; height: 200px;
    background: #eef;
}

.badge {
    position: absolute;          /* out of the normal flow */
    top: 10px; right: 10px;      /* measured from the parent */
    background: crimson; color: white;
    padding: 4px 10px;
}

.moved {
    position: relative;
```

```
    top: 20px; left: 40px;       /* shifted, place still reserved */
}
```

UNIT 5 — CSS LAYOUT TECHNIQUES

■ Flexbox — One-Dimensional Layouts

Flexbox is a one-dimensional layout system: it arranges the children of a container **in one direction only** — a row or a column — and distributes space among them. The container gets **display : flex;** and grows **flex properties**: **flex-direction** (row / column; row is the default), **justify-content** (distribution along the main axis: flex-start, center, space-between, space-around), **align-items** (alignment on the cross axis: stretch, center, flex-start), **flex-wrap** (whether the items wrap to a next line), and **gap** for the space between the items. The children get **flex : 1** (grow to fill the free space equally), **flex : 0 0 200px** (fixed width) and **order** (to change the visual order). Flexbox is the best choice for navigation bars, button rows, centring an element and "cards in a row".

```
.row {
    display: flex;
    flex-direction: row;       /* main axis = horizontal */
    justify-content: space-between;
    align-items:   center;
    gap: 16px;
}

.grow { flex: 1; }             /* take all remaining space */
.fixed { flex: 0 0 200px; }    /* always exactly 200px     */

.center-anything {
    display: flex;
    justify-content: center;   /* horizontal centring */
    align-items:     center;   /* vertical centring   */
    height: 100vh;
}
```

■ CSS Grid — Row and Column Layouts

Grid is the two-dimensional system: it divides the container into **rows and columns at the same time**, and each item may be placed into a given cell or span several of them. The container gets **display : grid;** and defines its tracks: **grid-template-columns : 200px 1fr 1fr;** (three columns; fr = fraction of the free space), **grid-template-rows**, **gap** and the shorthand **grid-template-areas** which names the regions. The children are placed with **grid-column : 1 / 3;** (from line 1 to line 3 — spans two columns) or with the area names. Grid is the best choice for the whole page skeleton, dashboards, galleries and forms where rows and columns must line up.

```
.page {
    display: grid;
```

```
    grid-template-columns: 220px 1fr;
    grid-template-rows:    70px 1fr 50px;
    grid-template-areas:
        "head  head"
        "side  main"
        "foot  foot";
    gap: 8px;
    height: 100vh;
}

header { grid-area: head; }
aside  { grid-area: side; }
main   { grid-area: main; }
footer { grid-area: foot; }
```

■ Flexbox vs Grid — Basic Examples of Arranging Content

	Flexbox	**CSS Grid**
Dimension	one direction at a time (row OR column)	rows AND columns together (two-dimensional)
Content-driven?	yes — items may push each other (flex : grow / shrink)	layout-driven — the track plan is declared first
Typical job	nav bars, toolbars, centring, rows of cards	page skeleton, dashboards, image galleries
Alignment tools	justify-content, align-items	the same properties on the container, plus justify-self / align-self and place-items

---

Two practical examples: **(1) three equal cards** — in Flexbox: .row{display:flex;gap:16px;} .card{flex:1;} ; in Grid: .row{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;} — both give three equal columns, but the Grid can also stagger heights and place a card across two columns with *grid-column : span 2;*. **(2) centring an element both ways** — Flexbox: display:flex; justify-content:center; align-items:center; Grid: display:grid; place-items:center; — and a **responsive layout** collapses automatically with *auto-fit* and *minmax()*: **grid-template-columns : repeat(auto-fit, minmax(200px, 1fr));** which makes as many 200px columns as fit and stretches or wraps as the window changes.

Remember : Rule of thumb: **Grid for the layout of the page, Flexbox for the parts inside it**; the two systems are complementary and are regularly used on the same page.